

PROGRAMOWANIE GIER
WYKŁAD 3 –
NARZĘDZIA DO OBRÓBKİ SPRITÓW,
SYSTEM ANIMACJI, OBSŁUGA
DŹWIĘKÓW

Bogumiła Hnatkowska

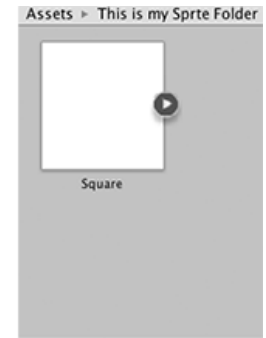
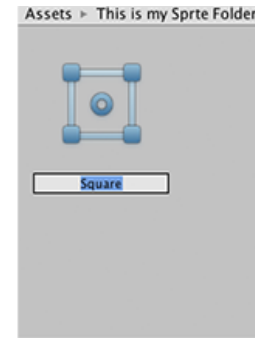
SPRITE (DUSZEK)

- Sprite – obiekt grafiki 2D, z którym można wchodzić w interakcje
- Podstawowe narzędzia:
 - Sprite Creator
 - Sprite Editor
 - Sprite Renderer



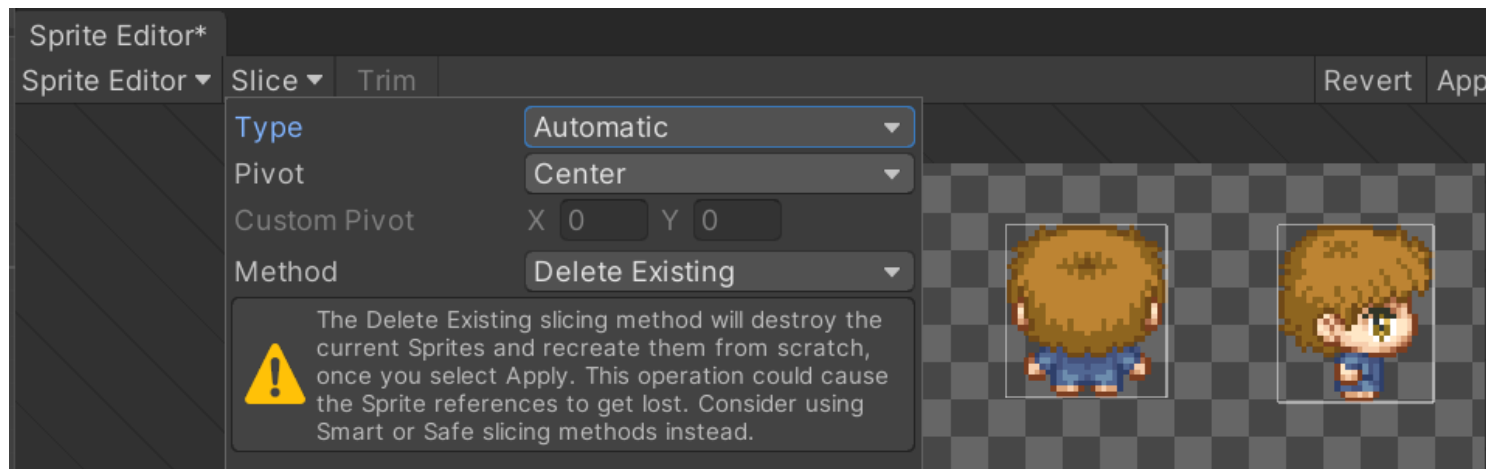
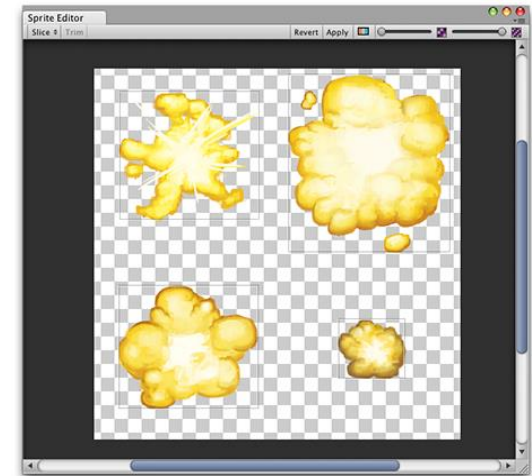
SPRITE CREATOR

- Służy do tworzenia obiektów zastępczych do grafik (kwadrat, trójkąt, ...), które można umieszczać na scenie
- Uruchomienie: Assets > Create > 2D > Sprites
- Zastąpienie obiektu zastępczego właściwym wymaga zmiany parametru Sprite w komponencie Renderer Component.



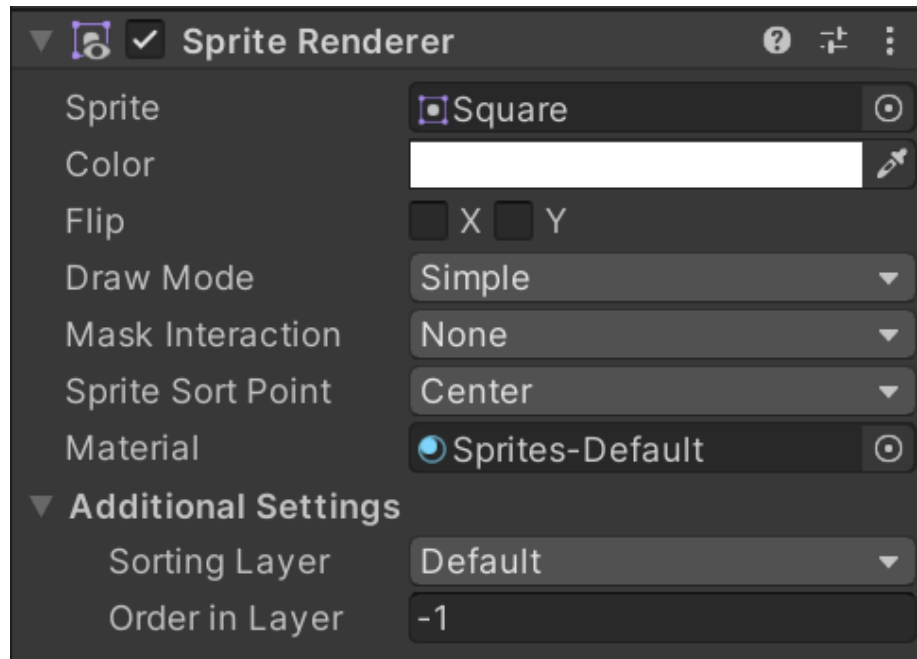
SPRITE EDITOR

- Pozwala „wyciąć” fragmenty grafik z obrazka na przykład na użytek animacji (warunek: tryb sprite ustawiony na Multiple)
- Tryby: ręczny, automatyczny, np. na podstawie parametrów siatki



WYŚWIETLACZ SPRITÓW (SPRITE RENDERER)

- Komponent, który umożliwia wyświetlanie spritów na scenach 2D i 3D



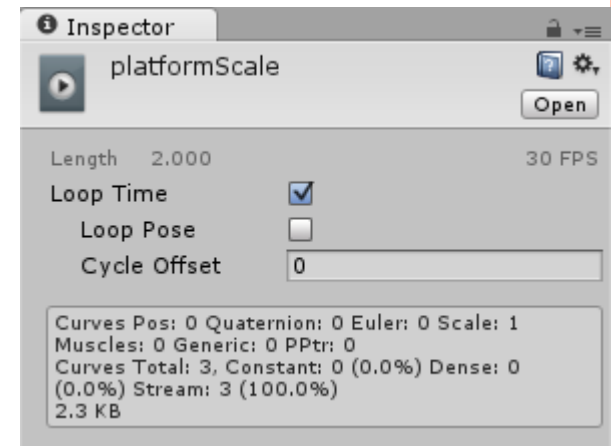
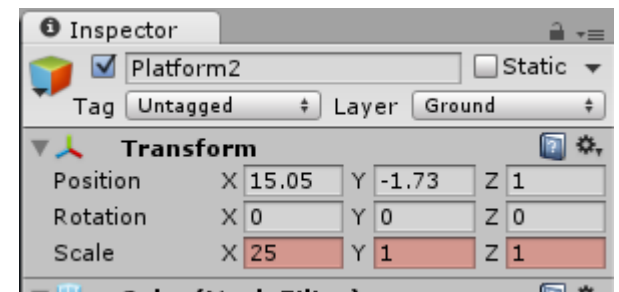
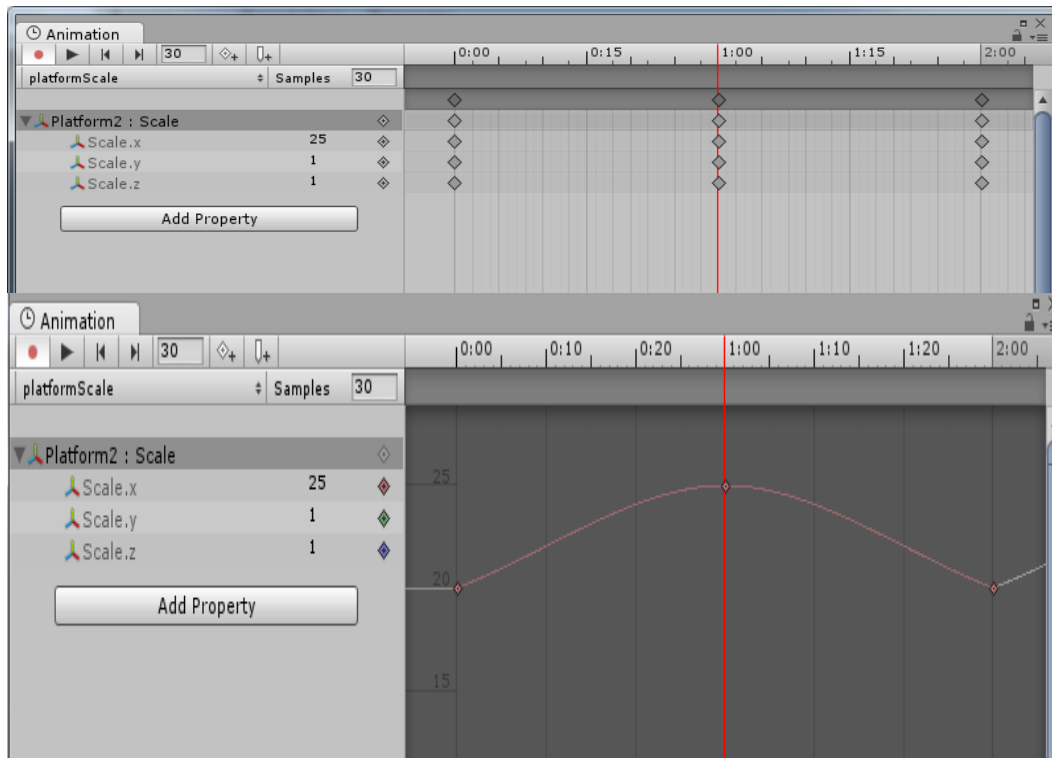
ANIMACJE – WPROWADZENIE

- Widok animacji pozwala definiować i parametryzować animacje w Unity dla wybranych obiektów (ich własności, np. Scale, Position)
- Dodanie animacji: Window/Animation/Animation
- Animacje można definiować w dwóch trybach:
 - Nagrywania (dopesheet)
 - Krzywych (curves)
- W obu trybach wartości zmienianej własności można podawać w oknie inspektora
- Dodatkowo, dla animacji można ustawić własności, np. czy ma się wykonywać w nieskończonej pętli



ANIMACJE – WPROWADZENIE, C.D.

- Przykład animacji dla własności Scale platformy



SYSTEM ANIMACJI (ANIMATION SYSTEM)

- Mecanim – nazwa systemu animacji w Unity
- Podstawowe elementy składowe:
 - Klipy animacji (animation clips)
 - Kontroler animacji (animation controller)
 - Komponent animatora (Animator component)



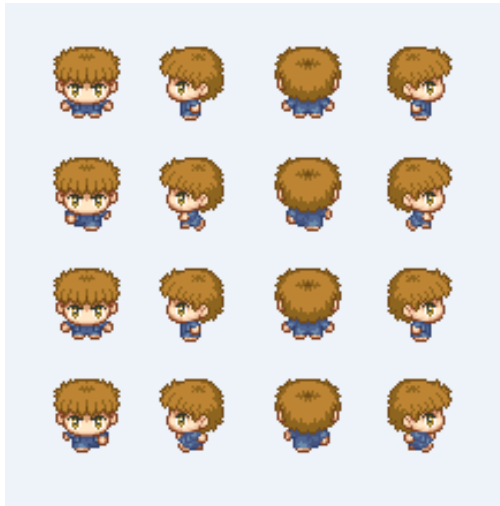
KLIP ANIMACJI (ANIMATION CLIP)

- Element systemu animacji unity – najmniejsza cegiełka, z której składa się animacja w Unity; izolowany fragment ruchu, np. bieg w lewo
- Zawiera informację jak poszczególne obiekty mają zmieniać swoje parametry w czasie
 - Importowany z zewnętrznych źródeł, np. 3DS Max, Maya, inne
 - Przygotowany w Unity:
 - Animacja pozycji, rotacji i skali
 - Kolor materiału, natężenie światła, głośność dźwięku
 - Własności skryptów typów int, float, Vector lub boolean
 - Możliwość wołania własnych funkcji w określonych momentach animacji



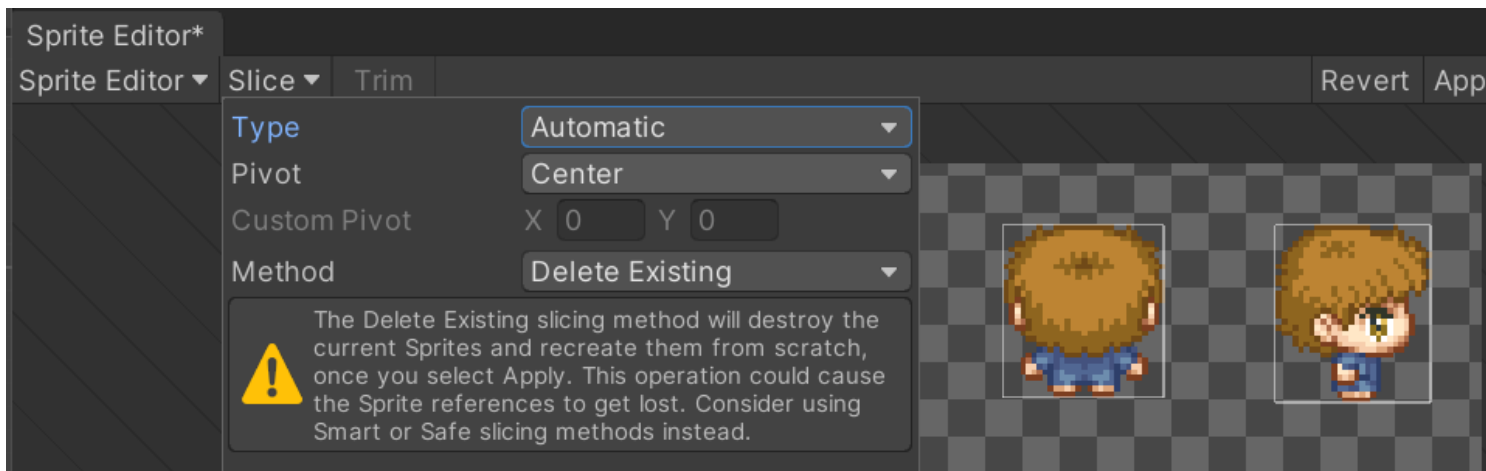
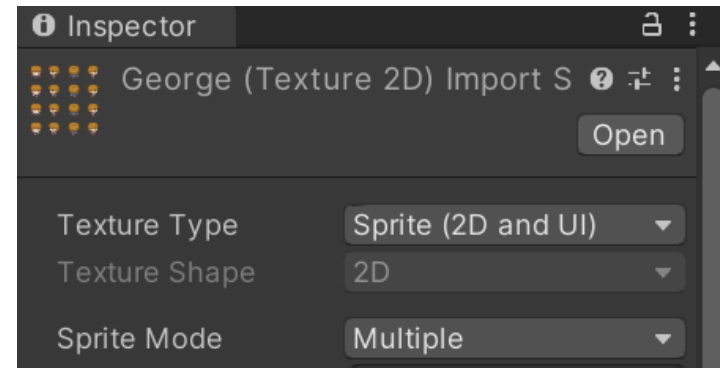
KLIP ANIMACJI – ANIMACJA POKLATKOWA

- Potrzebujemy grafik, które będą naszą postacią. Odpowiednie duszki można ściągnąć z opengameart.org (np. [george.png](#))



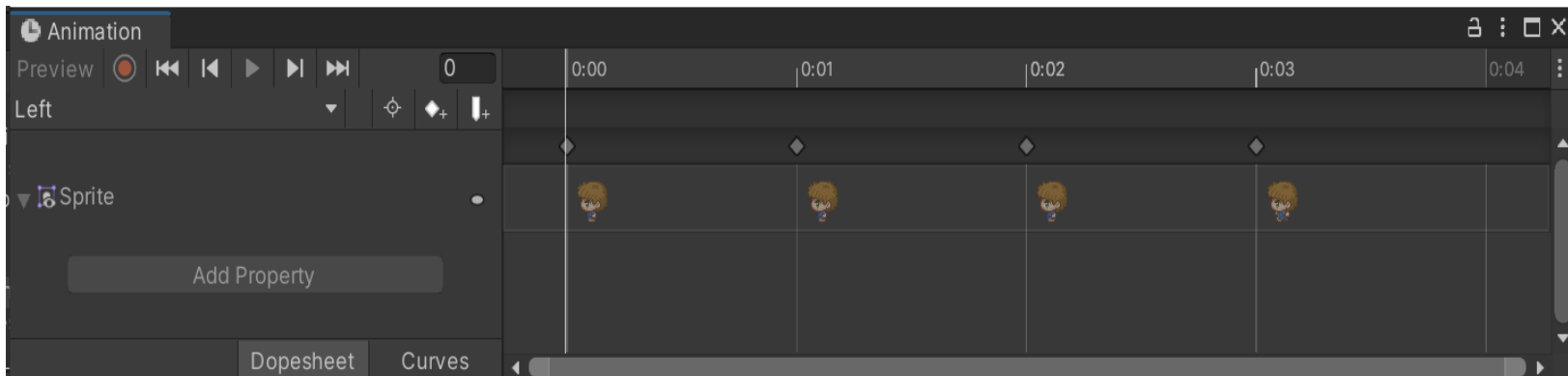
KLIP ANIMACJI – ANIMACJA POKLATKOWA, C.D.

- Zmień ustawienia trybu grafiki na Sprite, Multiple
- Otwórz edytor i podziel arkusz używając np. Automatic/Trim



KLIP ANIMACJI – ANIMACJA POKLATKOWA

- Otwórz okno Animacji (Window > Animation).
Utwórz nową animację wskazując, które duszki będą w niej brały udział, i w których momentach (frame), np. dla animacji Left (ruch w lewo)

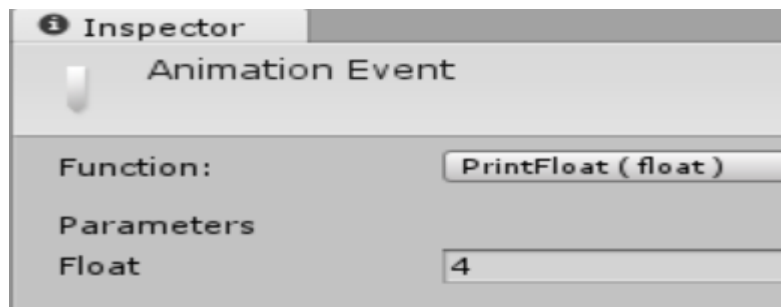


ZDARZENIA ANIMACJI

- Animacja może wysyłać zdarzenia w określonych punktach czasu, obsługiwane przez funkcję dołączoną do animowanego obiektu
- Funkcja może być bezparametrowa lub mieć jeden parametr typu: float, string, int, object

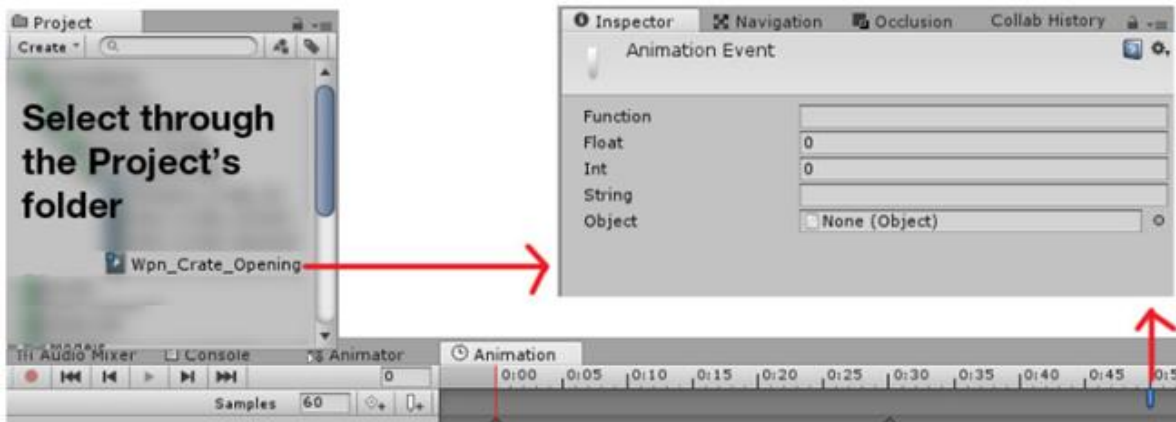
```
public void PrintFloat(float theValue)
{
    Debug.Log("PrintFloat is called with a value of " + theValue);
}
```

- Dla zdarzenia można określić wysyłany parametr

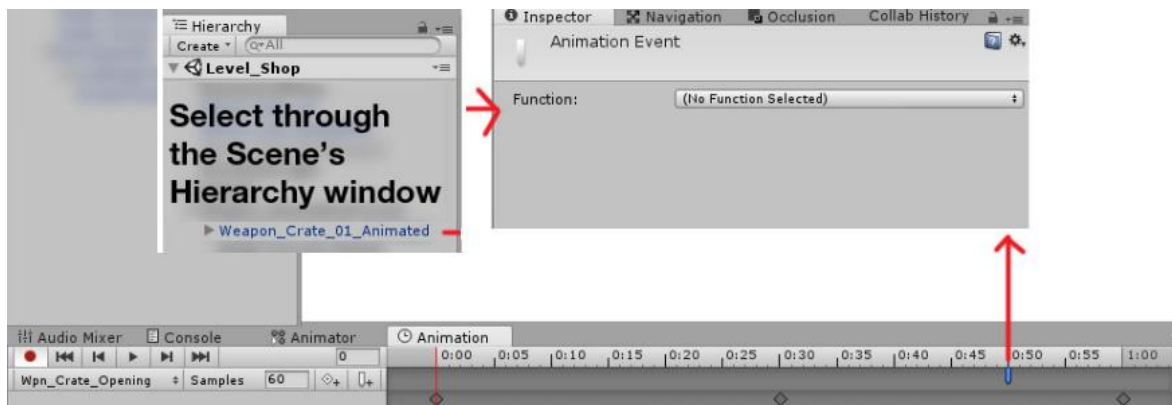


ZDARZENIA ANIMACJI, C.D.

- Aby parametry były widoczne

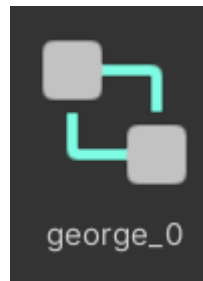


- Parametry niewidoczne



KONTROLER ANIMACJI (ANIMATION CONTROLLER)

- Zasób, który przechowuje referencje do klipów z animacjami i przełącza je z wykorzystaniem maszyny stanów
- Jest dołączany do komponentu Animatora (dla obiektu gry)

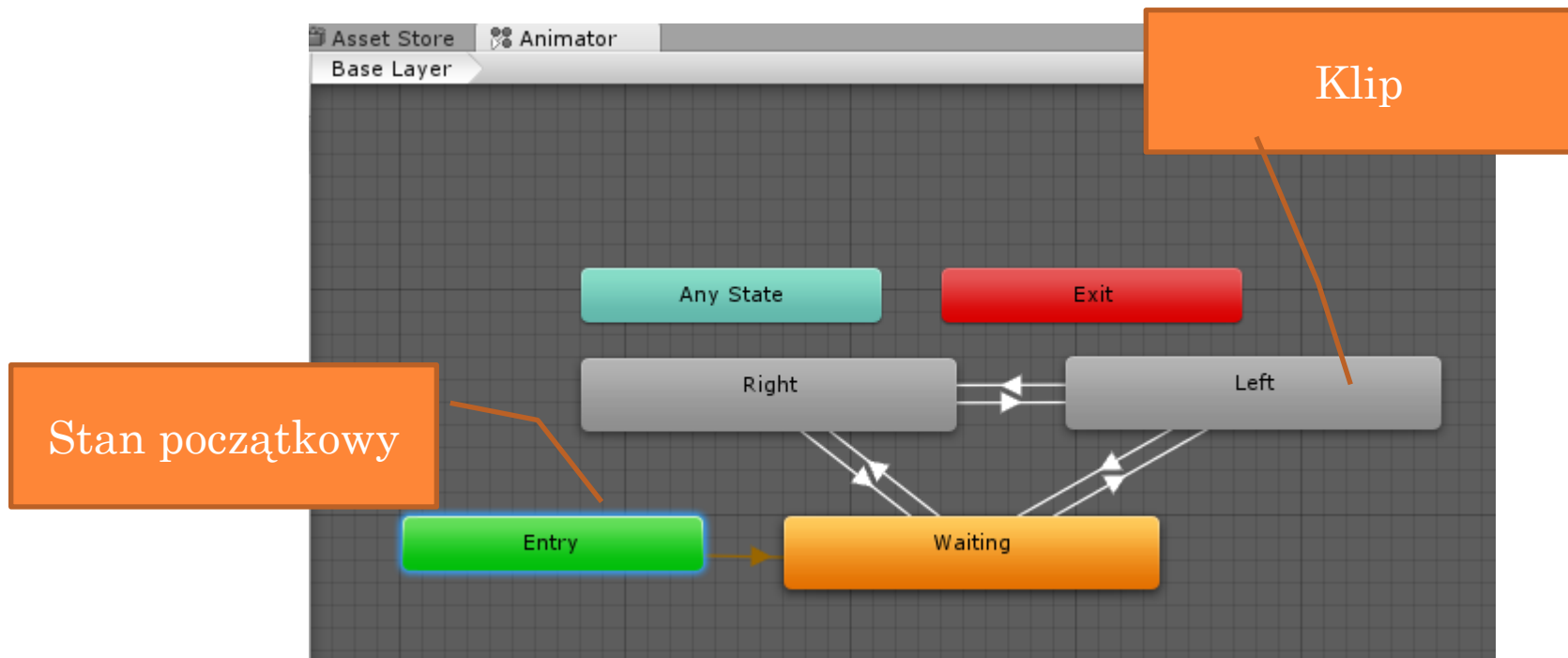


- Tworzenie:
 - Okno zasobów: Create > Animator Controller
 - Przy tworzeniu animacji (efekt uboczny)



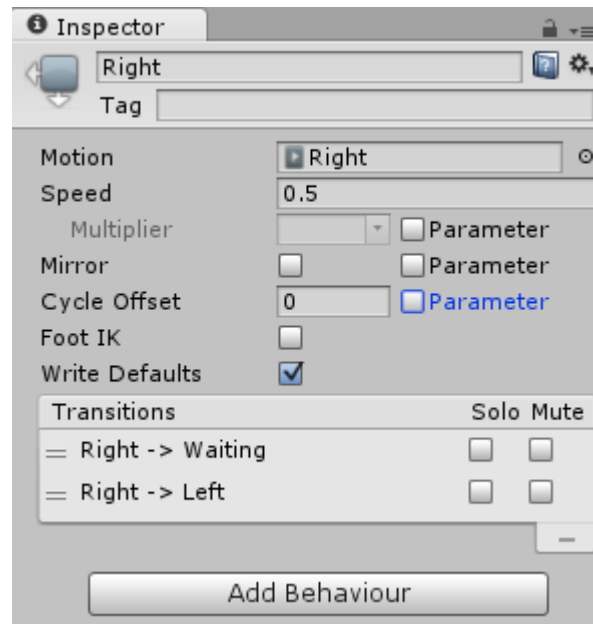
KONTROLER ANIMACJI – MASZYNA STANÓW

- Stan reprezentuje klip z animacją
- Zielony – stan początkowy; Czerwony – stan końcowy
- Any state – „skrót notacyjny”; może mieć tylko wychodzące tranzycje



DEFINICJA STANU

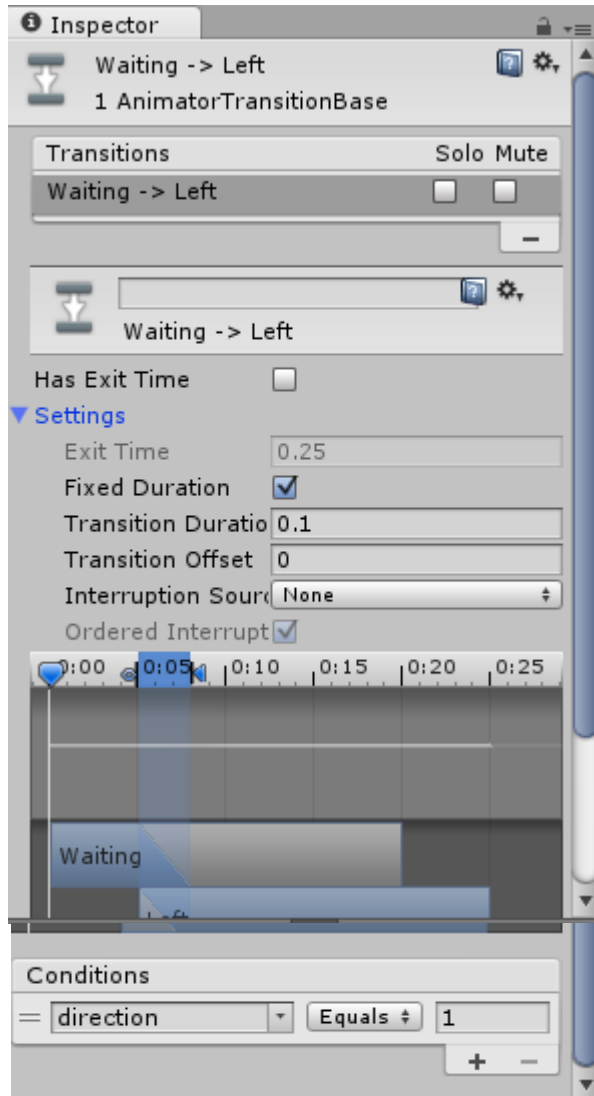
○ Parametry stanu:



- Motion - klip z animacją
- Speed – prędkość animacji
- Transitions – wychodzące tranzycje



PARAMETRY TRANZYCJI



Preview playback current time

Duration "in"

Transition Duration

Duration "out"



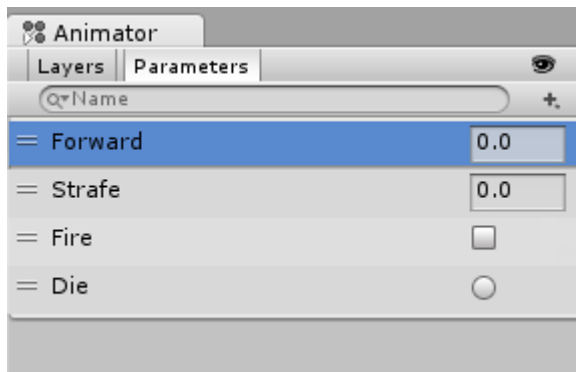
Current State

Transition Offset

Target State

PARAMETRY ANIMACJI

- Zmienne, które są zdefiniowane w kontrolerze animacji i są dostępne z poziomu skryptów:
 - Int
 - Float
 - Bool
 - Trigger (okrągły przycisk)



```
// Use this for initialization
void Start () {
    animator = GetComponent<Animator>();
}

// Update is called once per frame
void Update () {
    float h = Input.GetAxis("Horizontal");
    float v = Input.GetAxis("Vertical");
    bool fire = Input.GetButtonDown("Fire1");

    animator.SetFloat("Forward",v);
    animator.SetFloat("Strafe",h);
    animator.SetBool("Fire", fire);
}

void OnCollisionEnter(Collision col) {
    if (col.gameObject.CompareTag("Enemy"))
    {
        animator.SetTrigger("Die");
    }
}
```

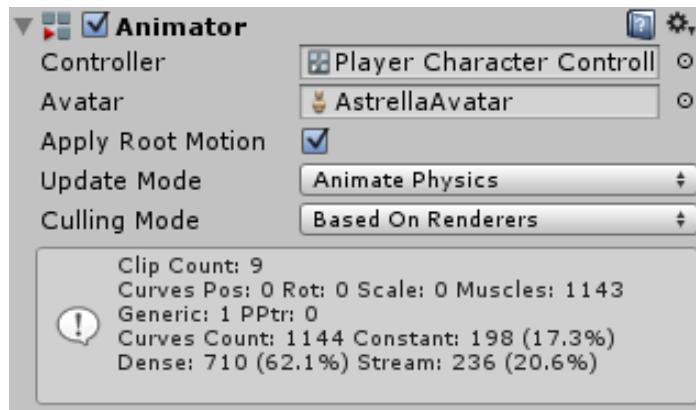
ZACHOWANIE MASZINY

- Zachowanie obiektu w każdym ze stanów może być zmieniane przez nadpisanie metod w klasie dziedziczącej po `StateMachineBehaviour`
- Do skryptu można przejść wybierając w oknie Animatora stan, a następnie w oknie Inspektora „Add Behaviour”

```
public class WaitingBehaviour : StateMachineBehaviour {  
  
    // OnStateEnter is called when a transition starts and the state  
    //override public void OnStateEnter(Animator animator, AnimatorStateInfo stateInfo, int transitionIndex)  
    //  
    //}  
  
    // OnStateUpdate is called on each Update frame between OnStateEnter and OnStateExit  
    //override public void OnStateUpdate(Animator animator, AnimatorStateInfo stateInfo, int transitionIndex)  
    //  
    //}  
  
    // OnStateExit is called when a transition ends and the state machine enters another state  
    //override public void OnStateExit(Animator animator, AnimatorStateInfo stateInfo, int transitionIndex)  
    //  
    //}  
  
    // OnStateMove is called right after Animator.OnAnimatorMove(). Code to move the position of the  
    //override public void OnStateMove(Animator animator, AnimatorStateInfo stateInfo, int transitionIndex)  
    //  
    //}  
  
    // OnStateIK is called right after Animator.OnAnimatorIK(). Code to move the position of the  
    //override public void OnStateIK(Animator animator, AnimatorStateInfo stateInfo, int transitionIndex)  
    //  
    //}  
}
```

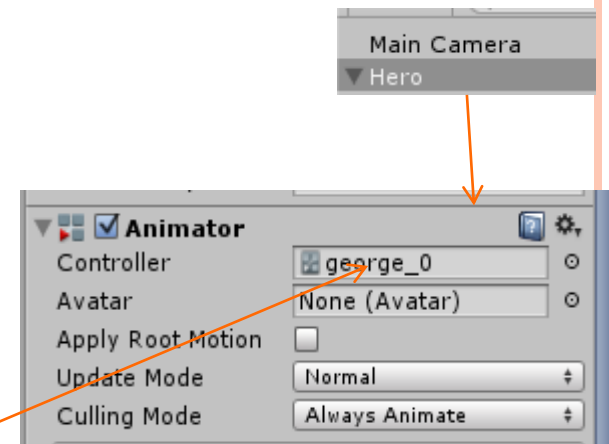
KOMPONENT ANIMATORA (ANIMATOR COMPONENT)

- Służy do powiązania kontrolera animacji z obiektem gry
- Wymaga ustalenia referencji do kontrolera animacji, który definiuje, które klipy używać i steruje przejściami między nimi



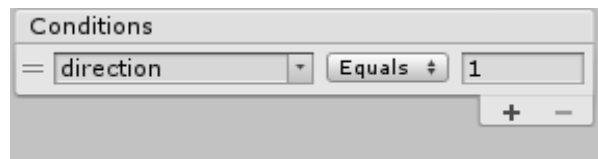
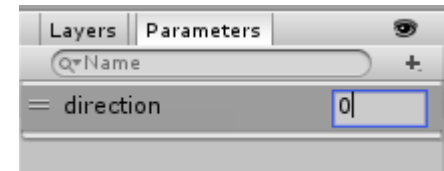
PRZYKŁAD - KLIPY Z ANIMACJAMI POSTACI

- Przygotuj klipy animacji dla różnych typów ruchów (w lewo, w prawo, czekanie, ...)
- Dla każdej animacji Unity tworzy Kontroler animacji, chyba że stworzysz kolejne klipy w oknie Animation
- Połączymy animacje w jedną maszynę stanów, jak pokazano na slajdzie 14



PRZYKŁAD - DEFINICJA PARAMETRÓW I TRANZYCJI

- W oknie Animatora zdefiniowano parametr direction (0 – waiting, 1 – left, 2 – right)
- Wykorzystano parametr direction do zdefiniowania warunków przejść między stanami:



STEROWANIE POSTACIĄ (LEWO, PRAWO, BEZCZYNNOŚĆ)

```
public class playerController : MonoBehaviour {  
    public float maxSpeed = 10f;    // maksymalna prędkość poruszania się postaci  
    private Animator animator;      // komponent animatora  
    private Rigidbody2D myRigidbody; // postać  
  
    // Use this for initialization  
    void Start () {  
        animator = this.GetComponent<Animator>();  
        myRigidbody = this.GetComponent<Rigidbody2D> ();  
    }  
  
    void FixedUpdate () {            // wykorzystanie zmiennej direction do sterowania  
        var horizontal = Input.GetAxis("Horizontal");  
  
        if (horizontal > 0) {  
            animator.SetInteger ("direction", 2);  
        } else if (horizontal < 0) {  
            animator.SetInteger ("direction", 1);  
        } else {  
            animator.SetInteger ("direction", 0);  
        }  
  
        myRigidbody.velocity = new Vector2 (maxSpeed * horizontal, myRigidbody.velocity.y);  
    }  
}
```


OBSŁUGA SKOKÓW

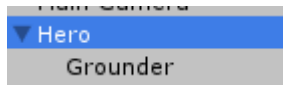
```
public float maxForce = 300f;    // siła "skakania"

void Update(){
    if (Input.GetKeyDown (KeyCode.Space)) {
        myRigidbody.AddForce (new Vector2 (0, maxForce));
    }
}
```

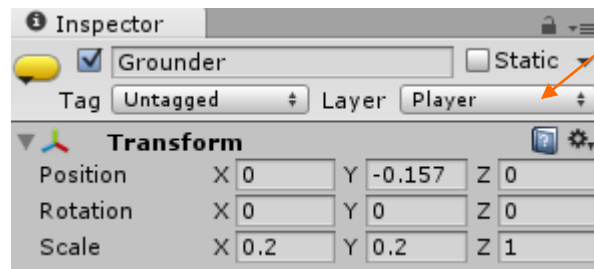


ZABEZPIECZENIE PRZED SKAKANIEM BEZ ODBICIA OD PODŁOŻA Z WYKORZYSTANIEM WARSTW

- Dodajemy pusty pod-obiekt do postaci (grounder)



- Pozycjonujemy, aby był na wysokości „stóp”; definiujemy, że jest na warstwie Player



ZABEZPIECZENIE PRZED SKAKANIEM BEZ ODBICIA OD PODŁOŻA, C.D.

- Modyfikujemy skrypt kontrolujący postać:

```
public Transform groundCheck; // komponent, który sprawdza, czy dotykamy platformy
public LayerMask whatIsGround; // Everything except Player
private float groundRadius = 0.3f; // promień, w którym szukamy podłoża
private bool onTheGround = true; // true - gdy dotykamy czegoś z wyjątkiem gracza

void Update(){
    // jeżeli postać dotyka podłoża i spacja
    if (onTheGround && Input.GetKeyDown (KeyCode.Space)) {
        onTheGround = false;
        myRigidbody.AddForce (new Vector2 (0, maxForce));
    }
}

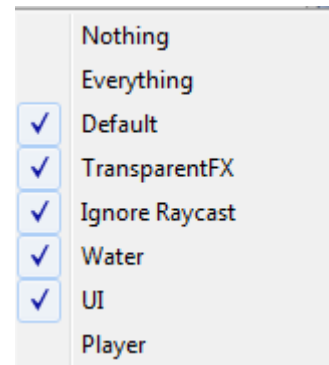
// Update is called once per frame
void FixedUpdate () {
    var horizontal = Input.GetAxis("Horizontal");

    // Sprawdź, czy komponent postaci (groundCheck) dotyka czegoś, co nie jest graczem
    onTheGround = Physics2D.OverlapCircle (groundCheck.position, groundRadius, whatIsGround);
```



ZABEZPIECZENIE PRZED SKAKANIEM BEZ ODBICIA OD PODŁOŻA, C.D.

- Ustawienie parametrów skryptu (podłożem jest wszystko z wyjątkiem obiektów oznaczonych jako Player)



WSPIERANE FORMATY DŹWIĘKÓW

- Nieskompresowane: WAV (Windows), AIF (MacOS)
- Skompresowane: MP3, OGG
- Inne: MOD, XM



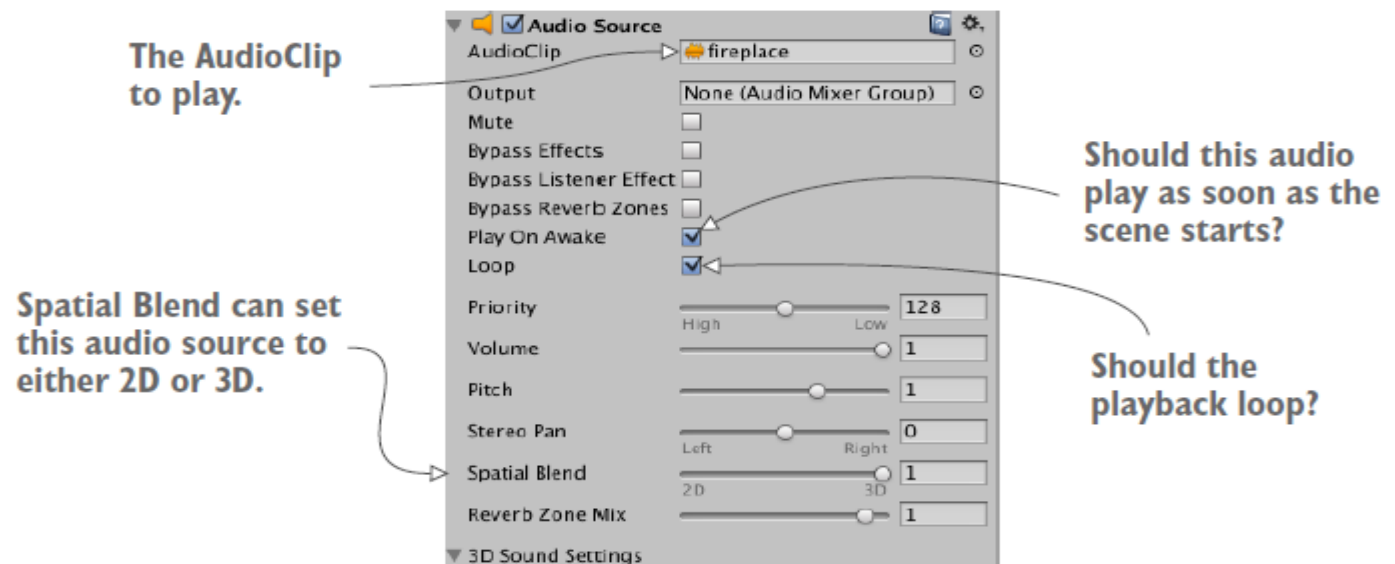
ODGRYWANIE PLIKÓW DŹWIĘKOWYCH

- Klip audio (Audio clip) – zasób (plik) dźwiękowy
- Źródło dźwięku (Audio Source) – obiekt, który jest źródłem dźwięku; jego położenie jest ważne przy dźwiękach w grze 3D
- Słuchacz (Audio Listener) – obiekt, z którego perspektywy dźwięk jest słyszalny



ODGRYWANIE DŹWIĘKU (2D) W PĘTLI

- Wybierz źródło dźwięku (np. postać wroga lub nowy, pusty obiekt)
- Dodaj komponent Audio > Audio Source i ustaw mu właściwy klip
- Zmień ustawienia: Play On Awake, Looping



URUCHAMIANIE ODTWARZANIA W KODZIE

- Odtwarzanie dźwięku przy uderzeniu w przeszkodę

```
private AudioSource audioSource;
public AudioClip hitSound;

void Start () {
    audioSource = this.GetComponent<AudioSource>();
    audioSource.clip = hitSound;
}

private void OnCollisionEnter2D(Collision2D coll) {
    if (coll.gameObject.tag == "Enemy") {
        audioSource.PlayOneShot(hitSound);
        // audioSource.Play();
    }
}
```



STEROWANIE GŁOŚNOŚCIĄ DŹWIĘKU

```
public class VolumeController : MonoBehaviour {  
    public float volume = 0.5f;  
    public float minVolume = 0.0f;  
    public float maxVolume = 1.0f;  
  
    private AudioSource audio;  
  
    // Use this for initialization  
    void Start () {  
        audio = this.GetComponent<AudioSource> ();  
    }  
  
    // Update is called once per frame  
    void Update () {  
        audio.volume = volume;  
    }  
  
    void OnGUI(){  
        GUI.Label (new Rect (150, 0, 100, 30), "Volume");  
        volume = GUI.HorizontalSlider (new Rect (150, 30, 100, 30), volume, minVolume, maxVolume);  
    }  
}
```

NATYCHMIASTOWY TRYB GUI (IMMEDIATE MODE GUI)

- Niezależna cecha Unity w stosunku do obiektów gry opartych o system UI (sterowany kodem system GUI)
- Wykorzystywana przez programistów w trakcie rozwoju gry.
- W tym trybie obiekty są tworzone w ramach funkcji OnGUI
- Kod wyświetlający interfejs jest wykonywany w każdej ramce (dla aktywnych obiektów) i rysowany na ekranie



NATYCHMIASTOWY TRYB GUI (IMMEDIATE MODE GUI), C.D.

○ Podstawowe kontrolki:

- Etykieta (Label):

```
GUI.Label (new Rect (25, 25, 100, 30), "Label");
```

- Przycisk (Button):

```
if (GUI.Button (new Rect (25, 25, 100, 30), "Button")) {
```

```
    // Kod obsługi przycisku
```

```
}
```

- Powtarzalny przycisk (RepeatButton)

```
if (GUI.RepeatButton (new Rect (25, 25, 100, 30), "RepeatButton")) {
```

```
    // Kod uruchamiany w każdej ramce jeżeli trzymamy przycisk
```

```
}
```



NATYCHMIASTOWY TRYB GUI (IMMEDIATE MODE GUI), C.D.

○ Podstawowe kontrolki:

- Pole tekstowe (TextField)

```
private string textFieldString = "text field";  
textFieldString = GUI.TextField (new Rect (25, 25, 100, 30),  
textFieldString);
```

- Obszar tekstowy (TextArea)

```
private string textAreaString = "text area";  
textAreaString = GUI.TextArea (new Rect (25, 25, 100,  
30), textAreaString);
```

- Przełącznik (Toggle)

```
private bool toggleBool = true;  
toggleBool = GUI.Toggle (new Rect (25, 25, 100, 30),  
toggleBool, "Toggle");
```



NATYCHMIASTOWY TRYB GUI (IMMEDIATE MODE GUI), C.D.

○ Podstawowe kontrolki:

- Suwak poziomy / pionowy (Horizontal / Vertical Slider)
private float hSliderValue = 0.0f;
hSliderValue = GUI.HorizontalSlider (new Rect (25, 25, 100, 30), hSliderValue, 0.0f, 10.0f);
- Poziomy / pionowy pasek przewijania (Horizontal/Vertical Scrollbar)
private float hScrollbarValue;
hScrollbarValue = GUI.HorizontalScrollbar (new Rect (25, 25, 100, 30), hScrollbarValue, 1.0f, 0.0f, 10.0f);



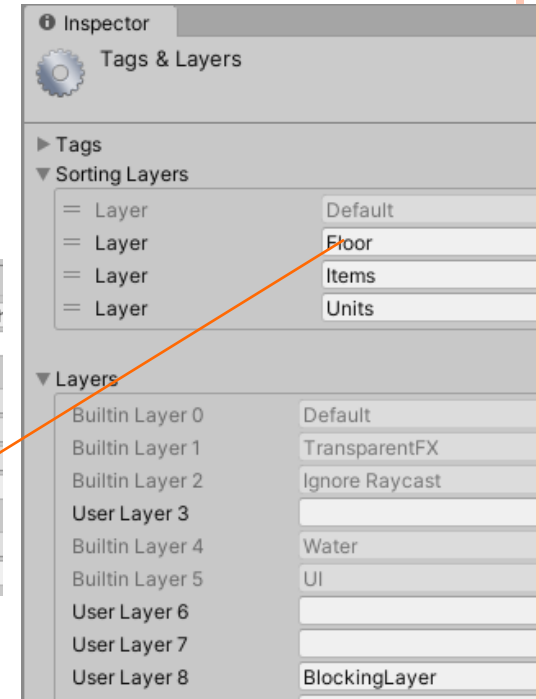
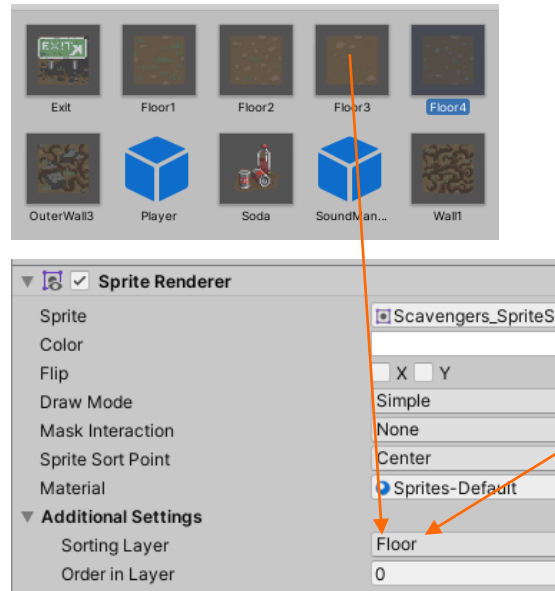
PROCEDURALNE GENEROWANIE ZAWARTOŚCI (PCG)

- Zastosowania PCG. Generacja:
 - poziomów
 - unikalnych przedmiotów
 - tekstur
 - modeli
 - fabuły
 - muzyki
- Generacja poziomów:
 - Świat zamknięty
 - Świat otwarty
 - Zawartość generuje się z różnych powodów: aby zapewnić unikalność, poprawić wydajność.



PCG – PRZYKŁAD

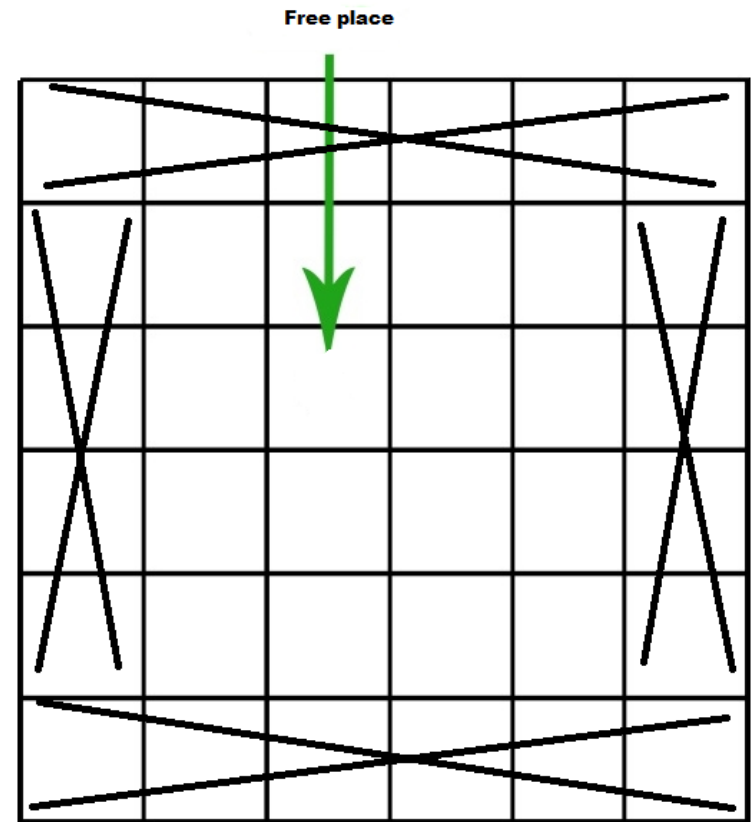
- Świat zamknięty, reprezentujący poziom (na podstawie tutoriala Unity)



PCG – PRZYKŁAD, C.D.

○ Idea algorytmu:

- Krok 1. Generacja planszy:
na zewnątrz ściany, w
środku podłoga
- Krok 2: Na wolnych
miejscach podłogi:
 - Krok 2.1: Przeszkody
 - Krok 2.2: Jedzenie
 - Krok 2.3: Przeciwnicy
 - Krok 2.4: Wyście



PCG – PRZYKŁAD (2D ROUGHLIKE), C.D.

```
[Serializable]
public class Count
{
    public int minimum;
    public int maximum;

    public Count (int min, int max)
    {
        minimum = min;
        maximum = max;
    }
}

public int columns = 8;           //Number of columns in our game board.
public int rows = 8;             //Number of rows in our game board.
public Count wallCount = new Count (5, 9); //Lower and upper limit for our random number of walls per level.
public Count foodCount = new Count (1, 5); //Lower and upper limit for our random number of food items per level.
public GameObject exit;          //Prefab to spawn for exit.
public GameObject[] floorTiles;  //Array of floor prefabs.
public GameObject[] wallTiles;  //Array of wall prefabs.
public GameObject[] foodTiles;  //Array of food prefabs.
public GameObject[] enemyTiles; //Array of enemy prefabs.
public GameObject[] outerWallTiles; //Array of outer tile prefabs.

private Transform boardHolder; //A reference to the transform of our Board object.
private List <Vector3> gridPositions = new List <Vector3> (); //A list of possible locations to place tiles.
```

PCG – PRZYKŁAD (2D ROUGHLIKE), C.D.

```
//Sets up the outer walls and floor (background) of the game board.
void BoardSetup ()
{
    boardHolder = new GameObject ("Board").transform;

    for(int x = -1; x < columns + 1; x++)
    {
        for(int y = -1; y < rows + 1; y++)
        {
            GameObject toInstantiate = floorTiles[Random.Range (0,floorTiles.Length)];

            if(x == -1 || x == columns || y == -1 || y == rows)
                toInstantiate = outerWallTiles [Random.Range (0, outerWallTiles.Length)];

            GameObject instance =
                Instantiate (toInstantiate, new Vector3 (x, y, 0f), Quaternion.identity) as GameObject;

            instance.transform.SetParent (boardHolder);
        }
    }
}
```



PCG – PRZYKŁAD (2D ROUGHLIKE), C.D.

//RandomPosition returns a random position from our list gridPositions.

```
Vector3 RandomPosition ()  
{  
    int randomIndex = Random.Range (0, gridPositions.Count);  
    Vector3 randomPosition = gridPositions[randomIndex];  
    gridPositions.RemoveAt (randomIndex);  
    return randomPosition;  
}
```

//LayoutObjectAtRandom places a random number (min-max) of random elements from tileArray.

```
void LayoutObjectAtRandom (GameObject[] tileArray, int minimum, int maximum)  
{  
    int objectCount = Random.Range (minimum, maximum+1);  
  
    for(int i = 0; i < objectCount; i++)  
    {  
        Vector3 randomPosition = RandomPosition();  
        GameObject tileChoice = tileArray[Random.Range (0, tileArray.Length)];  
        Instantiate(tileChoice, randomPosition, Quaternion.identity);  
    }  
}
```



PCG – PRZYKŁAD (2D ROUGHLIKE), C.D.

```
//SetupScene initializes our level and calls the previous functions to lay out the game board
public void SetupScene (int level)
{
    BoardSetup ();
    InitialiseList ();
    LayoutObjectAtRandom (wallTiles, wallCount.minimum, wallCount.maximum);
    LayoutObjectAtRandom (foodTiles, foodCount.minimum, foodCount.maximum);

    int enemyCount = (int)Mathf.Log(level, 2f);
    LayoutObjectAtRandom (enemyTiles, enemyCount, enemyCount);

    Instantiate (exit, new Vector3 (columns - 1, rows - 1, 0f), Quaternion.identity);
}
```



PCG – PRZYKŁAD 2 (2D ROUGHLIKE)

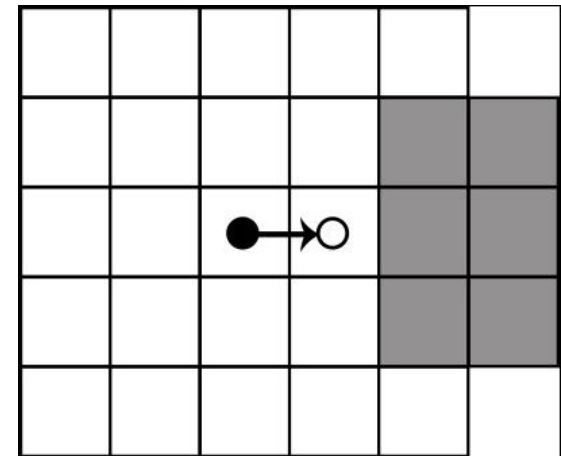
- Świat otwarty – generowany na podstawie ruchów gracza



PCG – PRZYKŁAD 2 (2D ROUGHLIKE)

○ Idea algorytmu:

- Wygeneruj bazową planszę, np. 5 x 5
- Ustaw gracza na środku
- Dopóki nie koniec poziomu:
 - Odczytaj kierunek ruchu gracza i ustal jego nowe położenie
 - Sprawdź, czy gracz widzi założoną (np. 6) liczbę kafelków w kierunku, w którym się porusza; Jeżeli nie – wygeneruj fragment planszy:
 - Podłoga
 - Losowo – przeszkoda, jedzenie, przeciwnik, wyjście (może być tylko 1 na planszy)



ELEMENTY WPŁYWAJĄCE NA STABILNOŚĆ DZIAŁANIA APLIKACJI

- Atrybuty
- Metoda OnValidate
- Assercje



ATRYBUT – REQUIRECOMPONENT

- RequireComponent(Type requiredComponent)
- Funkcja edytora; dodaje wymagany komponent do obiektu gry w momencie dodania do obiektu skryptu opatrzonego atrybutem

```
[RequireComponent(typeof(Rigidbody2D))]  
public class CubeController : MonoBehaviour  
{  
    private Rigidbody2D body;  
    ...  
  
    void Start() {  
        body = GetComponent<Rigidbody2D>();  
    }  
}
```



INNE UŻYTECZNE ATRYBUTY

- RangeAttribute(min, max) – Jego zastosowanie zmienia sposób wyświetlania zmiennej – suwak
- TooltipAttribute(tekst) – definicja podpowiedzi w Inspektorze

```
[RangeAttribute(10, 300)]  
[Tooltip("mph")]  
public int maxCarSpeed;
```



METODA ONVALIDATE

- Funkcja MonoBehaviour wywoływana TYLKO w edytorze podczas zmiany skryptu lub zmiany wartości serializowanych własności w Inspektorze; może drukować komunikaty lub ustawiać zmiennym pożądane wartości

```
public int velocity;  
void OnValidate() {  
    if (velocity % 2 != 0)  
        velocity--;  
}
```



ASERCJE

- Klasa Assert
 - AreApproximatelyEqual, AreNotApproximatelyEqual
 - IsNotNull, IsNull
 - IsTrue, IsNotTrue,
 - AreNotEqual, AreEqual
 - static bool raiseExceptions; - jeżeli ustawione na true – zgłoszony wyjątek `AssertionException`

```
public GameObject character;  
void Start() {  
    Assert.IsNotNull(character, "Character is null"); // Komunikat na konsoli  
}  
  
void Start() {  
    try { Assert.IsNotNull(character, "Character is null"); }  
    catch (AssertionException) {  
        Debug.Log("Exception raised");  
    }  
}
```



ASERCJE, C.D.

```
using UnityEngine;
using UnityEngine.Assertions;

public class AssertionExampleClass : MonoBehaviour {
    public int health;
    public GameObject go;

    void Update() {
        // You expect the health never to be equal to zero
        Assert.AreNotEqual(0, health);

        // The referenced GameObject should be always
        // (in every frame) be active
        Assert.IsTrue(go.activeInHierarchy);
    }
}
```

PYTANIA KONTROLNE

- Jakie są podstawowe elementy systemu Mecanim? Jakie istnieją pomiędzy nimi relacje?
- Czym może być warunkowane przejście na maszynie stanów w kontrolerze animacji?
- Wymień podstawowe elementy pozwalające na odtwarzanie dźwięków w Unity?
- Czym jest IMGUI? Jaka metoda trzeba przykryć, aby z tego trybu skorzystać?
- Czym jest PCG? Co może być generowane?

